ERP System Implementation Guide

Sales, Production Lines, and Accounting Modules

This comprehensive documentation covers the implementation of sales & inventory management, production lines & manufacturing, and accounting & financial management modules in an ERP system.

Table of Contents

1. Sales & Inventory Management Module
2. Production Lines & Manufacturing Module
3. Accounting & Financial Management Module
4. Treasury & Cash Management
5. Chart of Accounts & General Ledger
6. Item Cards & Inventory Tracking
7. Integration & System Architecture

Sales & Inventory Management Module

Core Concepts

The Sales & Inventory module handles the complete lifecycle of sales transactions, inventory management, and warehouse operations. It provides comprehensive treatment of sales concepts and inventory management, explaining how to manage stock and handle sales operations.

Key Components

1. Customer Management

```csharp
// Customer entity structure
public class Customer
{
    public int CustomerId { get; set; }
    public string CustomerCode { get; set; }
    public int CompanyId { get; set; } // com_code
    public string CustomerName { get; set; }
    public string ContactPerson { get; set; }
    public string Phone { get; set; }
    public string Email { get; set; }
    public string Address { get; set; }
    public decimal CreditLimit { get; set; }
    public decimal CurrentBalance { get; set; }
    public DateTime CreatedDate { get; set; }
    public bool IsActive { get; set; }
}
```

2. Sales Invoice Management

- **Invoice Processing**: Handle complete sales transaction lifecycle
- **Customer Balance Tracking**: Monitor customer payment history and outstanding amounts
- **Service Integration**: Include service details within customer invoices

```csharp
public class SalesInvoice
{
    public int InvoiceId { get; set; }
    public string InvoiceNumber { get; set; }
    public int CustomerId { get; set; }
    public DateTime InvoiceDate { get; set; }
    public decimal TotalAmount { get; set; }
    public decimal MoneyForAccount { get; set; } // money_for_account per invoice
    public decimal CustomerBalanceBefore { get; set; }
    public decimal CustomerBalanceAfter { get; set; }
    public decimal VatAmount { get; set; }
    public decimal DiscountAmount { get; set; }
    public List<SalesInvoiceItem> Items { get; set; }
    public List<ServiceItem> Services { get; set; }
}
```

3. Inventory Management

- **Multi-warehouse Support**: Manage inventory across multiple locations
- **Stock Transfer**: Handle inventory transfers between warehouses

- **Real-time Stock Tracking**: Monitor available quantities across all locations

```csharp
public class InventoryItem
{
    public int ItemId { get; set; }
    public string ItemCode { get; set; }
    public string ItemName { get; set; }
    public int WarehouseId { get; set; }
    public decimal AvailableQuantity { get; set; }
    public decimal ReservedQuantity { get; set; }
    public decimal UnitPrice { get; set; }
    public string BatchNumber { get; set; } // For batch tracking
    public DateTime ExpiryDate { get; set; }
}
```

4. Warehouse Transfer System

- **Transfer Requests**: Process requests to move inventory between warehouses
- **Transfer Validation**: Receiving warehouse validates quantities and items
- **Transfer Reporting**: Generate reports on transfer efficiency and timing

Implementation Features

Sales Processing Workflow

1. **Customer Selection**: Choose existing customer or create new customer record
2. **Item Selection**: Add items from inventory with real-time availability check
3. **Pricing & Discounts**: Apply customer-specific pricing and discount rules
4. **Payment Processing**: Handle various payment methods (cash, credit, installments)
5. **Invoice Generation**: Create invoice with proper numbering and formatting
6. **Inventory Update**: Automatically update stock levels across warehouses

Inventory Control Features

- **Batch Management**: Track items by batch numbers with expiry dates
- **Multi-unit Support**: Handle different units of measure (wholesale, retail)
- **Barcode Integration**: Support barcode scanning for quick item identification
- **Stock Alerts**: Generate alerts for low stock levels and expiring items

---

Production Lines & Manufacturing Module

Advanced Production Management

This module covers advanced production line management and internal workshop operations, contributing to enhanced knowledge in production management.

Core Components

1. Production Planning

```csharp
public class ProductionPlan
{
    public int PlanId { get; set; }
    public string PlanNumber { get; set; }
    public int ProductId { get; set; }
    public decimal PlannedQuantity { get; set; }
    public DateTime StartDate { get; set; }
    public DateTime EndDate { get; set; }
    public int ProductionLineId { get; set; }
    public ProductionStatus Status { get; set; }
    public List<ProductionResource> RequiredResources { get; set; }
}
```

2. Production Line Management

- **Line Configuration**: Define production line capabilities and capacity
- **Resource Allocation**: Assign materials, equipment, and labor to production lines
- **Scheduling**: Optimize production scheduling based on demand and resources

```csharp
public class ProductionLine
{
    public int LineId { get; set; }
    public string LineName { get; set; }
    public decimal Capacity { get; set; }
    public TimeSpan OperatingHours { get; set; }
    public List<Equipment> Equipment { get; set; }
    public List<WorkStation> WorkStations { get; set; }
    public bool IsActive { get; set; }
}
```

3. Bill of Materials (BOM)

- **Component Tracking**: Define all materials and components needed for production
- **Version Control**: Maintain different versions of BOMs for product changes
- **Cost Calculation**: Calculate production costs based on material and labor costs

```csharp
public class BillOfMaterials
{
    public int BomId { get; set; }
    public int ProductId { get; set; }
    public string Version { get; set; }
    public DateTime EffectiveDate { get; set; }
    public List<BomComponent> Components { get; set; }
    public decimal TotalCost { get; set; }
}
```

4. Work Order Management

- **Order Creation**: Generate work orders from sales orders or production plans
- **Progress Tracking**: Monitor production progress in real-time
- **Quality Control**: Integrate quality checkpoints throughout production

Manufacturing Features

Production Monitoring

- **Real-time Tracking**: Monitor production progress across all lines
- **Employee Shifts**: Track employee login/logout and performance impact
- **Performance Metrics**: Measure overall performance for each employee or unit
- **Automated Notifications**: Send alerts for production changes or delays

Quality Management

- **Quality Checkpoints**: Define quality control points throughout production
- **Defect Tracking**: Record and analyze production defects
- **Corrective Actions**: Implement corrective measures for quality issues

---

Accounting & Financial Management Module

Comprehensive Financial System

This module handles all accounting operations including journal entries, general ledger, and financial reporting.

Core Accounting Concepts

1. Journal Entries (اليوميات)

Journal entries are records of all financial transactions in the company, essential for transparency and financial accountability.

```csharp
public class JournalEntry
{
    public int JournalId { get; set; }
    public string JournalNumber { get; set; }
    public DateTime TransactionDate { get; set; }
    public string Description { get; set; }
    public string ReferenceNumber { get; set; }
    public decimal TotalDebit { get; set; }
    public decimal TotalCredit { get; set; }
    public List<JournalEntryLine> Lines { get; set; }
    public JournalStatus Status { get; set; }
}

public class JournalEntryLine
{
    public int LineId { get; set; }
    public int AccountId { get; set; }
    public decimal DebitAmount { get; set; }
    public decimal CreditAmount { get; set; }
    public string Description { get; set; }
}
```

2. General Ledger (دفتر الأستاذ)

The general ledger maintains detailed records of all accounts and their movements.

```csharp
public class GeneralLedger
{
    public int LedgerId { get; set; }
    public int AccountId { get; set; }
    public DateTime TransactionDate { get; set; }
    public string Description { get; set; }
    public decimal DebitAmount { get; set; }
    public decimal CreditAmount { get; set; }
    public decimal RunningBalance { get; set; }
    public string ReferenceNumber { get; set; }
}
```

Accounting Operations

1. Revenue Recognition

csharp

```csharp
// Cash Collection Entry
// Debit: Cash Account
// Credit: Sales Revenue or Accounts Receivable
public void RecordCashCollection(decimal amount, int customerId)
{
    var journalEntry = new JournalEntry
    {
        TransactionDate = DateTime.Now,
        Description = "Cash Collection",
        Lines = new List<JournalEntryLine>
        {
            new JournalEntryLine
            {
                AccountId = CashAccountId,
                DebitAmount = amount,
                Description = "Cash received"
            },
            new JournalEntryLine
            {
                AccountId = AccountsReceivableId,
                CreditAmount = amount,
                Description = "Customer payment"
            }
        }
    };
}
```

2. Expense Management

```csharp
// Cash Disbursement Entry
// Debit: Expense Account
// Credit: Cash Account
public void RecordCashDisbursement(decimal amount, int expenseAccountId)
{
    var journalEntry = new JournalEntry
    {
        TransactionDate = DateTime.Now,
        Description = "Cash Disbursement",
        Lines = new List<JournalEntryLine>
        {
            new JournalEntryLine
            {
                AccountId = expenseAccountId,
                DebitAmount = amount,
                Description = "Expense payment"
            },
            new JournalEntryLine
            {
                AccountId = CashAccountId,
                CreditAmount = amount,
                Description = "Cash paid"
            }
        }
    };
}
```

Financial Reporting

1. Trial Balance (ميزان المراجعة)

The trial balance ensures all debits equal credits across all accounts.

```csharp
public class TrialBalance
{
    public List<TrialBalanceLine> Lines { get; set; }
    public decimal TotalDebits { get; set; }
    public decimal TotalCredits { get; set; }
    public bool IsBalanced => TotalDebits == TotalCredits;
}

public class TrialBalanceLine
{
    public int AccountId { get; set; }
    public string AccountName { get; set; }
    public decimal DebitBalance { get; set; }
    public decimal CreditBalance { get; set; }
}
```

2. Balance Sheet (الميزانية)

```csharp
public class BalanceSheet
{
    public DateTime AsOfDate { get; set; }
    public List<BalanceSheetAccount> Assets { get; set; }
    public List<BalanceSheetAccount> Liabilities { get; set; }
    public List<BalanceSheetAccount> Equity { get; set; }

    public decimal TotalAssets => Assets.Sum(a => a.Amount);
    public decimal TotalLiabilitiesAndEquity =>
        Liabilities.Sum(l => l.Amount) + Equity.Sum(e => e.Amount);
}
```

Treasury & Cash Management

Cash Flow Management

The treasury module manages all cash operations, including multiple cash accounts and various types of financial movements.

Key Features

1. Multiple Cash Accounts

```csharp
public class CashAccount
{
    public int CashAccountId { get; set; }
    public string AccountName { get; set; }
    public string AccountNumber { get; set; }
    public decimal CurrentBalance { get; set; }
    public CashAccountType AccountType { get; set; } // Petty Cash, Bank, etc.
    public bool IsActive { get; set; }
}
```

2. Cash Movements

Handle various types of cash transactions:

- **Collections**: Cash received from customers
- **Disbursements**: Cash paid to suppliers or for expenses
- **Transfers**: Money movement between cash accounts

```csharp
public class CashMovement
{
    public int MovementId { get; set; }
    public int CashAccountId { get; set; }
    public DateTime TransactionDate { get; set; }
    public MovementType Type { get; set; } // In, Out, Transfer
    public decimal Amount { get; set; }
    public string Description { get; set; }
    public string ReferenceNumber { get; set; }
    public decimal BalanceBefore { get; set; }
    public decimal BalanceAfter { get; set; }
}
```

3. Cash Shift Management

Track cash handling by different users across shifts.

```csharp
public class CashShift
{
    public int ShiftId { get; set; }
    public int UserId { get; set; }
    public int CashAccountId { get; set; }
    public DateTime StartTime { get; set; }
    public DateTime? EndTime { get; set; }
    public decimal OpeningBalance { get; set; }
    public decimal ClosingBalance { get; set; }
    public decimal TotalReceipts { get; set; }
    public decimal TotalDisbursements { get; set; }
    public ShiftStatus Status { get; set; }
}
```

Treasury Operations

1. Daily Cash Summary

Generate daily reports showing all cash movements and current balances.

2. Cash Reconciliation

Compare system balances with physical cash counts to identify discrepancies.

3. Multi-Currency Support

Handle transactions in different currencies with proper exchange rate management.

Chart of Accounts & General Ledger

Account Structure

The chart of accounts represents the complete financial structure of the organization.

Account Hierarchy

1. Account Categories

```csharp
public enum AccountType
{
    Asset = 1,
    Liability = 2,
    Equity = 3,
    Revenue = 4,
    Expense = 5
}

public class Account
{
    public int AccountId { get; set; }
    public string AccountCode { get; set; }
    public string AccountName { get; set; }
    public AccountType Type { get; set; }
    public int? ParentAccountId { get; set; } // For hierarchical structure
    public bool IsParent { get; set; }
    public bool IsActive { get; set; }
    public decimal CurrentBalance { get; set; }
    public List<Account> ChildAccounts { get; set; }
}
```

2. Standard Account Structure

```plaintext
1000 - Assets
  1100 - Current Assets
    1110 - Cash and Cash Equivalents
    1120 - Accounts Receivable
    1130 - Inventory
  1200 - Fixed Assets
    1210 - Property, Plant & Equipment
    1220 - Accumulated Depreciation

2000 - Liabilities
  2100 - Current Liabilities
    2110 - Accounts Payable
    2120 - Accrued Expenses
  2200 - Long-term Liabilities
    2210 - Long-term Debt

3000 - Equity
  3100 - Owner's Equity
  3200 - Retained Earnings

4000 - Revenue
  4100 - Sales Revenue
  4200 - Service Revenue

5000 - Expenses
  5100 - Cost of Goods Sold
  5200 - Operating Expenses
```

Account Management Features

1. Unique Account Codes

Ensure each account has a unique identifier to prevent duplication.

2. Parent-Child Relationships

Support hierarchical account structure for better organization and reporting.

3. Account Balancing

Maintain accurate balances for all accounts with real-time updates.

Item Cards & Inventory Tracking

Comprehensive Item Management

The item card system tracks how each movement affects items and stores this information for effective inventory management.

Item Master Data

1. Item Information

```csharp
public class Item
{
    public int ItemId { get; set; }
    public string ItemCode { get; set; }
    public string ItemName { get; set; }
    public string Description { get; set; }
    public ItemType Type { get; set; } // Product, Service, Raw Material
    public string Barcode { get; set; }
    public bool TrackBatches { get; set; }
    public bool TrackSerial { get; set; }
    public List<ItemUnit> Units { get; set; }
    public decimal StandardCost { get; set; }
    public decimal SellingPrice { get; set; }
}
```

2. Multi-Unit Support

Handle items with multiple units of measure (e.g., wholesale boxes, retail pieces).

```csharp
public class ItemUnit
{
    public int UnitId { get; set; }
    public int ItemId { get; set; }
    public string UnitName { get; set; }
    public decimal ConversionFactor { get; set; } // 0.274 for retail unit example
    public bool IsBaseUnit { get; set; }
    public decimal UnitPrice { get; set; }
}
```

3. Batch Tracking

For items requiring batch management (expiry dates, lot numbers).

```csharp
public class ItemBatch
{
    public int BatchId { get; set; }
    public int ItemId { get; set; }
    public string BatchNumber { get; set; }
    public DateTime ExpiryDate { get; set; }
    public decimal AvailableQuantity { get; set; }
    public decimal UnitCost { get; set; }
    public int SupplierId { get; set; }
}
```

Movement Tracking

1. Inventory Movements

Track all item movements with complete audit trail.

```csharp
public class InventoryMovement
{
    public int MovementId { get; set; }
    public int ItemId { get; set; }
    public int WarehouseId { get; set; }
    public DateTime MovementDate { get; set; }
    public MovementType Type { get; set; } // In, Out, Transfer, Adjustment
    public decimal Quantity { get; set; }
    public decimal UnitCost { get; set; }
    public string ReferenceNumber { get; set; }
    public string Description { get; set; }
    public int? BatchId { get; set; }
    public decimal BalanceBefore { get; set; }
    public decimal BalanceAfter { get; set; }
}
```

2. Real-time Balance Updates

Maintain accurate inventory balances across all warehouses.

3. Cost Tracking

Support various costing methods (FIFO, LIFO, Average Cost).

Integration & System Architecture

System Integration Points

1. Sales-Inventory Integration

- Automatic inventory updates when sales are processed
- Real-time availability checking during sales
- Reserved quantity management for pending orders

2. Production-Inventory Integration

- Material requirement planning based on production schedules
- Automatic inventory consumption for production
- Finished goods receipt into inventory

3. Accounting Integration

- Automatic journal entries for all transactions
- Real-time general ledger updates
- Integrated financial reporting

Data Consistency

1. Transaction Management

Use database transactions to ensure data consistency across modules.

2. Audit Trail

Maintain complete audit trail for all system transactions.

3. User Permissions

Implement role-based access control to ensure data security.

Performance Optimization

1. Indexing Strategy

Optimize database performance with proper indexing on frequently queried fields.

2. Caching

Implement caching for frequently accessed reference data.

3. Background Processing

Use background jobs for heavy processing tasks like report generation.

Implementation Guidelines

1. Development Phases

1. **Phase 1**: Core data models and basic CRUD operations
2. **Phase 2**: Business logic and validation rules
3. **Phase 3**: Integration between modules
4. **Phase 4**: Reporting and analytics
5. **Phase 5**: Advanced features and optimization

2. Testing Strategy

- Unit tests for all business logic
- Integration tests for module interactions
- Performance tests for high-volume scenarios
- User acceptance testing for workflows

3. Deployment Considerations

- Database migration scripts
- Configuration management
- Monitoring and logging
- Backup and recovery procedures

4. Training and Documentation

- User manuals for each module
- Training programs for different user roles
- Technical documentation for system administrators
- API documentation for integrations

Conclusion

This comprehensive ERP system implementation covers all aspects of sales, production, and accounting operations. The modular design ensures flexibility and scalability while maintaining data integrity and system performance. Regular monitoring and continuous improvement will ensure the system meets evolving business needs.